

Meridian Overhaul Implementation Plan

Twenty-six-task delivery program across seven slices

MERIDIAN

Project record - August 26, 2026

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (- []) syntax for tracking.

Goal: Replace SimpleCrew's overlapping feature pages with Meridian's responsive Today, Plan, Activity, and Accounts workspaces backed by a normalized financial graph, unified Commitments, explainable transaction AI, and proposal-only financial automation.

Architecture: Build four independently releasable vertical slices. Introduce new domain modules and versioned SQLite migrations behind compatibility adapters; do not expand app.py with new business logic. The browser consumes stable /api/meridian/* read models while existing provider and Crew mutation code remains isolated until parity gates permit legacy removal.

Tech Stack: Python 3.9+, Flask, SQLite, vanilla JavaScript ES2020 modules, Jinja templates, CSS custom properties, pytest, Playwright browser smoke tests, Docker, GitHub Actions.

Spec: docs/superpowers/specs/2026-08-26-meridian-product-overhaul-design.md

Global Constraints

- Crew credentials and provider secrets remain server-side and never enter browser payloads, AI prompts, logs, or fixtures.
- Financial mutations are never retried automatically; uncertain outcomes require state verification.
- AI may update local classifications and forecasts automatically. Creating Commitments, changing funding rules, modifying Crew state, or moving money requires an explained proposal and owner approval.
- Mobile exposes the same core capabilities as desktop through responsive composition and progressive disclosure.
- Existing data migrations are non-destructive, versioned, idempotent, and resumable.
- Every normalized record retains provider provenance, freshness, and external identifiers.
- User rules and corrections outrank AI classifications.
- All new UI meets WCAG 2.2 AA contrast, keyboard, screen-reader, reduced-motion, safe-area, and 44 CSS-pixel touch-target requirements.
- Each slice must pass unit, integration, migration, API, and browser tests before the next slice begins.

Program sequence

- **1. Slice 1 - Trustworthy foundation and shell:** safety gates, CI, schema infrastructure, normalized reads, design system, navigation shell, Today summary, Activity ledger, and transaction inspector.
- **2. Slice 2 - Commitments and funding:** typed Commitment model, migration review, funding rules, Plan views, schedule proposals, and core Beacon coverage.
- **3. Slice 3 - Unified providers and transaction intelligence:** adapter normalization, reconciliation, deterministic rules, AI assignments, review workflow, and correction learning.
- **4. Slice 4 - Advanced intelligence and consolidation:** scenarios, contextual advisor, richer forecasts, complete responsive parity, legacy removal, and production hardening.
- **5. Slice 5 - Document Intelligence:** evidence storage, read-only email ingestion, safe document extraction, and bill/charge/payment reconciliation.
- **6. Slice 6 - Life Context:** opt-in calendar, payroll, travel, and shared-money evidence feeding explicit forecast assumptions.
- **7. Slice 7 - Asset and Contract Memory:** receipts, warranties, contracts, renewals, maintenance, and long-horizon reserves.

Slice 1 - Trustworthy foundation and shell

Task 1: Establish executable quality and production-safety gates

Files:

- Modify: `app.py:8044-8048`
- Modify: `requirements.txt`
- Modify: `.github/workflows/docker-image.yml`
- Create: `requirements-dev.txt`
- Create: `tests/test_production_config.py`
- Create: `tests/browser/test_smoke.py`

Interfaces:

- Consumes: existing Flask app and Docker image.
- Produces: create_app-compatible production settings, repeatable pytest command, and CI gates required by every later task.
- **[] Step 1: Write failing production-config tests**

```
def test_production_debug_is_disabled(simplecrew):
    assert simplecrew.app.debug is False

def test_session_cookie_defaults(simplecrew):
    assert simplecrew.app.config["SESSION_COOKIE_HTTPONLY"] is True
    assert simplecrew.app.config["SESSION_COOKIE_SAMESITE"] == "Lax"
```

- **[] Step 2: Run the focused test and record the failure**

Run: `pytest tests/test_production_config.py -q`

Expected: FAIL because the executable entry point starts Flask with debug mode enabled.

- **[] Step 3: Separate development and production launch behavior**

Add `gunicorn==23.0.0` to `requirements.txt`, put `pytest`, `playwright`, `ruff`, and `pip-audit` in `requirements-dev.txt`, configure secure cookie defaults, and change the Docker command to:

```
CMD ["gunicorn", "--bind", "0.0.0.0:8080", "--workers", "1", "--threads", "4", "app:app"]
```

Keep direct `python app.py` development-only and set `debug=os.getenv("FLASK_DEBUG") == "1"`.

- **[] Step 4: Add CI test, audit, Docker-build, and browser-smoke jobs**

Install production and development requirements, run `ruff check`, `pytest -q`, `pip-audit`, build the image, start it with an isolated database, and run `pytest tests/browser -q`. The publish job must depend on every gate and run only for main pushes.

- **[] Step 5: Verify the complete gate**

Run: `docker build -t meridian:test . && docker run --rm -v "$PWD:/src" -w /src meridian:test sh -lc 'pip install -r requirements-dev.txt && pytest -q'`

Expected: all tests pass; no development server or debugger is exposed.

- **[] Step 6: Commit**

```
git add -- app.py Dockerfile requirements.txt requirements-dev.txt .github/workflows/docker-image.yml
tests/test_production_config.py tests/browser/test_smoke.py
git commit -m "chore: enforce Meridian quality and production gates"
```

Task 2: Make action execution atomic and idempotent

Files:

- Modify: crew/actions.py
- Modify: crew/executors.py
- Modify: tests/crew/test_actions.py
- Modify: tests/crew/test_executors.py
- Modify: tests/test_app_crew_integration.py

Interfaces:

- Consumes: ActionStore, ExecutorSpec, and /api/actions//execute.
- Produces: ActionStore.claim_for_execution(request_id: str, execution_key: str) -> dict and exactly-once local execution semantics.
- **[] Step 1: Write a concurrent execution regression test**

Use two threads and a barrier to call `execute_approved_action` for the same approved request. Assert the executor call count is one and the second result is a conflict or the already-recorded terminal result.

- **[] Step 2: Run the regression test**

Run: `pytest tests/crew/test_executors.py -k concurrent -q`

Expected: FAIL with two executor calls.

- **[] Step 3: Add the execution claim state**

Add `EXECUTING` to `ActionState`, migrate `action_requests` with `execution_key` and `execution_started_at`, and claim with a single `BEGIN IMMEDIATE` transaction:

```
updated = conn.execute(
    "UPDATE action_requests SET state=?, execution_key=?, execution_started_at=? "
    "WHERE id=? AND state=?",
    ("executing", execution_key, _now(), request_id, "approved"),
)
if updated.rowcount != 1:
    raise IllegalTransitionError("Action is not available for execution")
```

- **[] Step 4: Execute only after a successful claim**

Generate or accept an idempotency key before calling the Crew executor. Preserve `uncertain_write` as a non-retryable failed outcome with `verify_state=True`.

- **[] Step 5: Run action and integration suites**

Run: `pytest tests/crew/test_actions.py tests/crew/test_executors.py tests/test_app_crew_integration.py -q`

Expected: PASS with one executor invocation under concurrency.

- **[] Step 6: Commit**

```
git add -- crew/actions.py crew/executors.py tests/crew/test_actions.py tests/crew/test_executors.py
tests/test_app_crew_integration.py
git commit -m "fix: claim approved actions before execution"
```

Task 3: Introduce versioned migrations and normalized read models

Files:

- Create: meridian/__init__.py
- Create: meridian/db.py
- Create: meridian/models.py
- Create: meridian/migrations/001_financial_graph.sql
- Create: meridian/repository.py
- Create: tests/meridian/test_migrations.py
- Create: tests/meridian/test_repository.py

Interfaces:

- Produces: run_migrations(db_path: str) -> list[str], FinancialRepository.upsert_account, upsert_transaction, list_accounts, list_transactions, and immutable DTOs AccountRecord and TransactionRecord.

- **[] Step 1: Write migration idempotency and rollback tests**

Assert two migration runs produce one schema_migrations entry and preserve existing legacy tables and rows.

- **[] Step 2: Write repository contract tests**

Verify provider/external-id uniqueness, freshness timestamps, JSON-safe DTO output, pagination, account filtering, and stable transaction ordering by occurred_at DESC, id DESC.

- **[] Step 3: Run tests and confirm missing-module failures**

Run: `pytest tests/meridian/test_migrations.py tests/meridian/test_repository.py -q`

- **[] Step 4: Implement the migration runner and schema**

Create normalized provider_connections, financial_accounts, financial_transactions, and transaction_relations tables. Use unique keys (provider, external_id) and never store provider credentials in these tables.

- **[] Step 5: Implement focused repository methods**

Use parameterized SQL, explicit transactions, cursor pagination, and `INSERT ... ON CONFLICT DO UPDATE`. Keep Flask and provider calls outside this module.

- **[] Step 6: Run focused and full suites**

Run: `pytest tests/meridian -q && pytest -q`

- **[] Step 7: Commit**

```
git add -- meridian tests/meridian
git commit -m "feat: add Meridian normalized financial read model"
```

Task 4: Adapt existing Crew data into the financial graph

Files:

- Create: meridian/providers/base.py
- Create: meridian/providers/crew.py
- Create: meridian/sync.py
- Create: tests/meridian/providers/test_crew.py
- Create: tests/meridian/test_sync.py
- Modify: app.py

Interfaces:

- Produces: ProviderSnapshot, NormalizedAccount, NormalizedTransaction, CrewReadAdapter.fetch_snapshot(), and sync_provider(adapter, repository) -> SyncReport.

- **[] Step 1: Write adapter tests from sanitized Crew fixtures**

Assert cents become decimal currency values, raw account numbers and bearer tokens are absent, transaction statuses are preserved, and owned-account transfers produce relation hints rather than spending categories.

- **[] Step 2: Write sync idempotency and partial-failure tests**

Two identical snapshots must not duplicate rows. A failed transaction page must retain prior trustworthy data and return SyncReport(status="partial").

- **[] Step 3: Run tests and confirm failures**

Run: `pytest tests/meridian/providers/test_crew.py tests/meridian/test_sync.py -q`

- **[] Step 4: Implement protocol and Crew adapter**

Define ProviderAdapter with provider_name and fetch_snapshot(). Wrap existing read functions; do not move Crew mutation code.

- **[] Step 5: Implement idempotent synchronization**

Store a sync run, upsert accounts and transactions, mark connection freshness only after a complete snapshot, and preserve the previous freshness marker on partial failure.

- **[] Step 6: Wire synchronization at the existing refresh boundary**

Call the adapter after a successful Crew read without changing current response shapes. Log counts and status, never payloads.

- **[] Step 7: Verify**

Run: `pytest tests/meridian tests/test_app_crew_integration.py -q`

- **[] Step 8: Commit**

```
git add -- meridian/providers meridian/sync.py tests/meridian app.py
git commit -m "feat: normalize Crew reads into Meridian"
```

Task 5: Build stable Meridian read APIs

Files:

- Create: meridian/api.py
- Create: meridian/services/today.py
- Create: meridian/services/activity.py
- Create: tests/meridian/test_api.py
- Create: tests/meridian/services/test_today.py
- Modify: app.py

Interfaces:

- Produces: GET /api/meridian/today, GET /api/meridian/activity, GET /api/meridian/transactions/, and GET /api/meridian/accounts.
- **[] Step 1: Write authenticated API contract tests**

Assert unauthenticated requests redirect or return 401 consistently; payloads include data_freshness, never contain credential fields, and use stable error objects {code, message, recovery_action}.

- **[] Step 2: Write Today calculation tests**

For a fixed account/transaction fixture, assert total cash, safe-to-spend inputs, the explicit upcoming_events: [] empty state, and stale-data labeling. Do not invent a forecast when the normalized graph is incomplete.

- **[] Step 3: Run focused tests**

Run: `pytest tests/meridian/test_api.py tests/meridian/services/test_today.py -q`

- **[] Step 4: Implement services and a Flask Blueprint**

Services consume only FinancialRepository. Register meridian_api under /api/meridian; keep response serialization in api.py and calculations in service modules.

- **[] Step 5: Verify payload safety and compatibility**

Run: `pytest tests/meridian -q && pytest -q`

- **[] Step 6: Commit**

```
git add -- meridian/api.py meridian/services tests/meridian app.py
git commit -m "feat: expose Meridian read APIs"
```

Task 6: Establish Meridian design tokens and responsive application shell

Files:

- Create: static/css/meridian/tokens.css
- Create: static/css/meridian/shell.css
- Create: static/css/meridian/motion.css
- Create: static/js/meridian/shell.js
- Create: templates/meridian/index.html
- Create: templates/meridian/partials/navigation.html
- Modify: app.py

- Modify: templates/base.html
- Create: tests/browser/test_meridian_shell.py

Interfaces:

- Produces: /meridian, MeridianShell.setWorkspace(name), desktop rail, mobile navigation, optional inspector rail, theme tokens, motion primitives, and safe-area layout.

☐ Step 1: Write browser tests for layout and navigation

At 1440×900 assert the compact rail, central canvas, and inspector region exist. At 390×844 assert all four workspaces are reachable, touch targets are at least 44 pixels, no horizontal document overflow exists, and no feature link is removed.

☐ Step 2: Run the browser tests

Run: `pytest tests/browser/test_meridian_shell.py -q`

Expected: FAIL because /meridian does not exist.

☐ Step 3: Define the visual tokens

Create explicit light and dark variables for warm canvas, elevated surface, ink, muted ink, borders, healthy/caution/risk/information, serif display, sans UI, spacing, radii, elevation, and motion durations. Do not inherit the old neon palette.

☐ Step 4: Implement the shell

Render Today, Plan, Activity, and Accounts as semantic links with `aria-current`. Preserve active workspace in the URL. Desktop uses grid columns `72px minmax(0,1fr) auto`; mobile uses a top identity bar and bottom workspace dock.

☐ Step 5: Add motion and accessibility controls

Use `prefers-reduced-motion`, visible focus rings, skip navigation, inert closed sheets, Escape handling, focus restoration, and safe-area insets.

☐ Step 6: Verify desktop and mobile

Run: `pytest tests/browser/test_meridian_shell.py -q`

☐ Step 7: Commit

```
git add -- static/css/meridian static/js/meridian templates/meridian templates/base.html app.py
tests/browser/test_meridian_shell.py
git commit -m "feat: add Meridian responsive application shell"
```

Task 7: Implement Today and the Activity ledger

Files:

- Create: static/js/meridian/api.js
- Create: static/js/meridian/today.js
- Create: static/js/meridian/activity.js
- Create: templates/meridian/partials/today.html
- Create: templates/meridian/partials/activity.html
- Create: static/css/meridian/workspaces.css

- Modify: tests/browser/test_meridian_shell.py
- Create: tests/browser/test_activity.py

Interfaces:

- Consumes: Meridian read APIs from Task 5.
- Produces: loadToday(), loadActivity({cursor, accountId}), date-grouped ledger, freshness indicators, and stable selection state.

[] Step 1: Write Today and ledger browser tests

Mock API responses and assert safe-to-spend explanation inputs render, stale data is labeled, transactions group by local date, amounts use correct signs, and loading/error/empty states preserve layout.

[] Step 2: Run tests and confirm failure

Run: `pytest tests/browser/test_meridian_shell.py tests/browser/test_activity.py -q`

[] Step 3: Implement a typed fetch wrapper and view controllers

meridianFetch(path, options) must enforce JSON responses, normalize API errors, support AbortController, and never retry mutations. Today and Activity controllers abort stale requests on workspace changes.

[] Step 4: Render the Editorial Wealth hierarchy

Use one dominant figure, directly labeled supporting values, compact upcoming events, and restrained status color. Avoid one-card-per-value layouts.

[] Step 5: Verify responsive and error states

Run: `pytest tests/browser/test_meridian_shell.py tests/browser/test_activity.py -q`

[] Step 6: Commit

```
git add -- static/js/meridian static/css/meridian/workspaces.css templates/meridian/partials tests/browser
git commit -m "feat: build Meridian Today and Activity workspaces"
```

Task 8: Build the complete transaction inspector

Files:

- Create: static/js/meridian/transaction-inspector.js
- Create: templates/meridian/partials/transaction-inspector.html
- Create: static/css/meridian/inspector.css
- Modify: templates/meridian/partials/activity.html
- Create: tests/browser/test_transaction_inspector.py

Interfaces:

- Consumes: GET /api/meridian/transactions/.
- Produces: openTransactionInspector(id), closeTransactionInspector(), desktop side inspector, mobile full-screen detail, URL-addressable selection, and contextual advisor launch metadata.

[] Step 1: Write complete-detail browser tests

Assert merchant, raw description, amount, date, status, account, an explicit Unassigned category, an explicit No Linked commitment state, recurrence, related transfers, provider/freshness, notes, and plan impact render. Test Escape, close-button focus restoration, deep-link reload, and 390-pixel viewport behavior.

- **[] Step 2: Run tests and confirm failure**

Run: `pytest tests/browser/test_transaction_inspector.py -q`

- **[] Step 3: Implement inspector state and semantic markup**

Update `?transaction=` without losing Activity filters. Use for facts, labeled sections for relationships and impact, and a dialog/full-page behavior appropriate to viewport.

- **[] Step 4: Add tactile behavior**

Create `static/js/meridian/haptics.js` with `haptic(kind)` supporting selection, success, and warning. Use `navigator.vibrate` only when supported and reduced motion is not requested; provide visual feedback regardless.

- **[] Step 5: Verify**

Run: `pytest tests/browser/test_transaction_inspector.py -q && pytest -q`

- **[] Step 6: Commit**

```
git add -- static/js/meridian static/css/meridian/inspector.css templates/meridian/partials
tests/browser/test_transaction_inspector.py
git commit -m "feat: add Meridian transaction inspector"
```

- **[] Slice 1 release gate**

Run: `ruff check . && pytest -q && docker build -t meridian:slice1 . && pytest tests/browser -q`

Manually verify authenticated desktop and mobile views against real read-only Crew data. Confirm old routes still work, no credentials appear in responses/logs, and no mutation is issued.

Slice 2 - Commitments and funding schedules

Task 9: Add the Commitment domain and migration review

Files:

- Create: `meridian/commitments.py`
- Create: `meridian/migrations/002_commitments.sql`
- Create: `meridian/migrate_legacy.py`
- Create: `tests/meridian/test_commitments.py`
- Create: `tests/meridian/test_legacy_migration.py`

Interfaces:

- Produces: `CommitmentType`, `Commitment`, `CommitmentRepository`, `preview_legacy_migration(db_path) -> MigrationPreview`, and `apply_legacy_migration(preview_id) -> MigrationReport`.
- **[] Step 1: Write domain invariant tests**

Cover bill due dates, optional goal dates, reserve cadence, buffer minimums, debt minimum payment, backing-pocket optionality, positive currency amounts, priorities, and archived status.

- **[] Step 2: Write non-destructive migration tests**

Legacy bills map to bill; ordinary goals to goal; known credit-card pockets to debt; ambiguous rows enter review. Assert source rows and Crew identifiers remain unchanged and reruns create no duplicates.

- **[] Step 3: Run focused tests**

Run: `pytest tests/meridian/test_commitments.py tests/meridian/test_legacy_migration.py -q`

- **[] Step 4: Implement schema, repository, preview, and apply operations**

Persist `migration_version`, `legacy_source`, `legacy_id`, and migration decisions. Require an explicit apply request; migration never invokes Crew.

- **[] Step 5: Verify and commit**

Run: `pytest tests/meridian -q`

```
git add -- meridian/commitments.py meridian/migrations/002_commitments.sql meridian/migrate_legacy.py
tests/meridian
git commit -m "feat: add unified Meridian Commitments"
```

Task 10: Implement funding-rule calculations

Files:

- Create: `meridian/funding.py`
- Create: `meridian/migrations/003_funding_rules.sql`
- Create: `tests/meridian/test_funding.py`

Interfaces:

- Produces: `FundingRule`, `FundingEvent`, and `project_funding(rule, commitment, cash_events, as_of) -> FundingProjection`.
- **[] Step 1: Write table-driven calculation tests**

Cover fixed-per-paycheck, paycheck percentage, calendar cadence, even-by-due-date, priority waterfall, min/max caps, pause, skip, one-time override, insufficient cash, and DST/date boundaries.

- **[] Step 2: Run tests and confirm missing implementation**

Run: `pytest tests/meridian/test_funding.py -q`

- **[] Step 3: Implement pure decimal-based calculations**

Use `Decimal` and explicit currency rounding. Return inputs, planned events, funded-by date, shortfall, and explanation factors. Do not call providers or mutate data.

- **[] Step 4: Add property tests for conservation and non-negative allocations**

Assert total allocations never exceed available cash or configured caps and identical inputs are deterministic.

- **[] Step 5: Verify and commit**

Run: `pytest tests/meridian/test_funding.py -q`

```
git add -- meridian/funding.py meridian/migrations/003_funding_rules.sql tests/meridian/test_funding.py
git commit -m "feat: calculate Meridian funding schedules"
```

Task 11: Generate approval-only schedule proposals

Files:

- Create: `meridian/funding_proposals.py`
- Modify: `crew/actions.py`
- Create: `tests/meridian/test_funding_proposals.py`
- Modify: `tests/crew/test_actions.py`

Interfaces:

- Produces: `propose_due_funding(projection, store, as_of) -> list[dict]` and action type `scheduled_move_money` using the existing reviewed executor path.
- ☐ **Step 1: Write proposal safety tests**

Assert a due event creates one idempotent proposal containing calculation, source, destination, amount, forecast effect, and rule id. Repeated triggers create no duplicates. No executor is called.

- ☐ **Step 2: Run tests and confirm failure**

Run: `pytest tests/meridian/test_funding_proposals.py -q`

- ☐ **Step 3: Implement proposal generation**

Use deterministic key `funding:::Route` accepted proposals through the atomic action pipeline from Task 2.

- ☐ **Step 4: Verify and commit**

Run: `pytest tests/meridian/test_funding_proposals.py tests/crew -q`

```
git add -- meridian/funding_proposals.py crew/actions.py tests/meridian/test_funding_proposals.py
tests/crew/test_actions.py
git commit -m "feat: propose scheduled funding safely"
```

Task 12: Build Plan, schedule editor, and core Beacon coverage

Files:

- Create: `meridian/services/plan.py`
- Modify: `meridian/api.py`
- Create: `static/js/meridian/plan.js`
- Create: `templates/meridian/partials/plan.html`
- Create: `static/css/meridian/plan.css`
- Create: `tests/meridian/services/test_plan.py`
- Create: `tests/browser/test_plan.py`

Interfaces:

- Produces: `/api/meridian/plan`, `/api/meridian/commitments`, `/api/meridian/funding-rules`, `command/timeline/allocation` views, and `proposal-preview` editor.

- **[] Step 1: Write Plan API and browser tests**

Cover all Commitment types, backing state, coverage, first shortfall, direct chart labels, synchronized views, every funding-rule editor mode, mobile steps, desktop live preview, and save-as-proposal behavior.

- **[] Step 2: Run tests and confirm failure**

Run: `pytest tests/meridian/services/test_plan.py tests/browser/test_plan.py -q`

- **[] Step 3: Implement Plan service and routes**

Return one canonical view model containing summary, timeline events, allocation segments, Commitments, freshness, and explanation factors.

- **[] Step 4: Implement responsive views and editor**

Keep one selected date range and Commitment across all views. Desktop uses side-by-side editor/preview; mobile uses ordered sheets with a sticky impact summary.

- **[] Step 5: Verify and commit**

Run: `pytest tests/meridian tests/browser/test_plan.py -q`

```
git add -- meridian/services/plan.py meridian/api.py static/js/meridian/plan.js static/css/meridian/plan.css
templates/meridian/partials/plan.html tests/meridian tests/browser/test_plan.py
git commit -m "feat: build Meridian Plan and funding editor"
```

- **[] Slice 2 release gate**

Run the full quality gate, migration twice against a production-data copy, and browser parity tests. Confirm migration creates no Crew mutation and schedule triggers create proposals only.

Slice 3 - Unified providers and transaction intelligence

Task 13: Normalize external providers and reconcile transfers

Files:

- Create: `meridian/providers/simplefin.py`
- Create: `meridian/providers/lunchflow.py`
- Create: `meridian/providers/splitwise.py`
- Create: `meridian/reconcile.py`
- Create: `tests/meridian/providers/test_external.py`
- Create: `tests/meridian/test_reconcile.py`

Interfaces:

- Produces provider adapters matching `ProviderAdapter` and `reconcile(snapshot, repository) -> ReconciliationReport`.
- **[] Step 1: Write sanitized adapter fixtures and contract tests**

Require stable external ids, provenance, freshness, account type/liability mapping, and redaction. Splitwise receivables become expected inflows and payables become Commitment candidates, not duplicate spending.

- **[] Step 2: Write reconciliation tests**

Cover owned-account transfer pairs, credit-card purchase/payment pairs, refunds, duplicate imports, pending-to-posted transitions, and ambiguous relationships.

- **[] Step 3: Run tests and confirm failure**

Run: `pytest tests/meridian/providers/test_external.py tests/meridian/test_reconcile.py -q`

- **[] Step 4: Implement adapters and deterministic reconciliation**

Keep provider calls in adapters and matching logic in `reconcile.py`. Record relations and confidence; never delete ambiguous records.

- **[] Step 5: Verify and commit**

Run: `pytest tests/meridian -q`

```
git add -- meridian/providers meridian/reconcile.py tests/meridian
git commit -m "feat: unify external accounts in Meridian"
```

Task 14: Add deterministic assignment rules and correction history

Files:

- Create: `meridian/classification.py`
- Create: `meridian/migrations/004_classification.sql`
- Create: `tests/meridian/test_classification.py`

Interfaces:

- Produces: `Classification`, `AssignmentRule`, `classify_deterministic(transaction, rules, commitments)`, and `record_correction(...)`.
- **[] Step 1: Write precedence and audit tests**

Assert user rule > transfer reconciliation > recurrence/merchant rule > fallback. Corrections preserve prior assignment and may create a normalized merchant/description/amount rule.

- **[] Step 2: Run tests and confirm failure**

Run: `pytest tests/meridian/test_classification.py -q`

- **[] Step 3: Implement deterministic pipeline**

Store category, Commitment, kind, confidence, explanation factors, provenance, and version. Ensure every transaction receives a fallback assignment.

- **[] Step 4: Verify and commit**

Run: `pytest tests/meridian/test_classification.py -q`

```
git add -- meridian/classification.py meridian/migrations/004_classification.sql
tests/meridian/test_classification.py
git commit -m "feat: add explainable transaction rules"
```

Task 15: Add provider-agnostic AI classification

Files:

- Create: meridian/ai/base.py
- Create: meridian/ai/classifier.py
- Create: meridian/ai/openai_compat.py
- Create: tests/meridian/ai/test_classifier.py
- Create: tests/meridian/ai/test_provider.py

Interfaces:

- Produces: `AIProvider.complete_json(schema, prompt)`, `AIClassifier.classify(batch)` -> `list[ClassificationSuggestion]`, and deterministic fallback on failure.
- **[] Step 1: Write prompt-redaction and schema tests**

Assert prompts contain only safe merchant, amount, date bucket, account type, candidate Commitments, and prior rules. Reject credentials, full account numbers, raw provider tokens, and free-form model output outside the JSON schema.

- **[] Step 2: Write failure and confidence tests**

Cover timeout, quota, malformed JSON, missing assignment, unsupported category, batching, and retry policy. Read calls may retry with bounded backoff; no mutation is involved.

- **[] Step 3: Run tests and confirm failure**

Run: `pytest tests/meridian/ai -q`

- **[] Step 4: Implement structured classification**

Send only transactions not resolved above the configured deterministic threshold. Persist model, provider, prompt version, confidence, evidence, latency, and token usage when supplied.

- **[] Step 5: Verify and commit**

Run: `pytest tests/meridian/ai tests/meridian/test_classification.py -q`

```
git add -- meridian/ai tests/meridian/ai
git commit -m "feat: classify Meridian transactions with AI"
```

Task 16: Build Review, Patterns, and enriched transaction details

Files:

- Modify: meridian/services/activity.py
- Modify: meridian/api.py
- Modify: static/js/meridian/activity.js
- Modify: static/js/meridian/transaction-inspector.js
- Create: static/js/meridian/review.js
- Create: tests/browser/test_transaction_review.py
- Modify: tests/browser/test_transaction_inspector.py

Interfaces:

- Produces: Activity modes timeline, review, patterns; correction API; assignment confidence and evidence display; contextual Commitment proposal.

- **[] Step 1: Write browser and API tests**

Cover low-confidence queue ordering, swipe and button alternatives, correction with optional rule creation, batch review, confidence labels, evidence display, recurrence and anomaly patterns, and preserved inspector selection.

- **[] Step 2: Run tests and confirm failure**

Run: `pytest tests/browser/test_transaction_review.py`
`tests/browser/test_transaction_inspector.py tests/meridian/test_api.py -q`

- **[] Step 3: Implement review and correction flow**

Corrections update local metadata atomically, append audit history, optionally create a rule, and immediately re-evaluate affected unreviewed transactions.

- **[] Step 4: Implement Patterns from deterministic aggregates**

Show recurrence, category shift, merchant trend, and cash-flow change with direct evidence links. AI may supply explanations but does not fabricate aggregates.

- **[] Step 5: Verify and commit**

Run: `pytest tests/meridian tests/browser -q`

```
git add -- meridian static/js/meridian tests
git commit -m "feat: add Meridian transaction review and patterns"
```

- **[] Slice 3 release gate**

Run full tests and a fixture-based reconciliation audit proving no double-counted owned transfers, credit payments, refunds, or Splitwise reimbursements. Confirm every transaction has an assignment and AI failure leaves synchronization healthy.

Slice 4 - Advanced intelligence and consolidation

Task 17: Expand Beacon into explainable scenarios

Files:

- Create: `meridian/beacon.py`
- Create: `meridian/scenarios.py`
- Create: `tests/meridian/test_beacon.py`
- Create: `tests/meridian/test_scenarios.py`
- Modify: `meridian/services/today.py`
- Modify: `meridian/services/plan.py`

Interfaces:

- Produces: `forecast(graph, commitments, rules, as_of) -> Forecast`, `run_scenario(base, changes) -> ScenarioResult`, and factors for every projection.
- **[] Step 1: Write forecast invariants and confidence tests**

Cover coverage horizons, runway, low point, first shortfall, variable ranges, stale/missing provider data, no-history behavior, and causal factor ids.

- **[] Step 2: Write scenario purity tests**

Changing contribution, due date, income, reserve, or expense returns a comparison without persisting any record.

- **[] Step 3: Run tests and confirm failure**

Run: `pytest tests/meridian/test_beacon.py tests/meridian/test_scenarios.py -q`

- **[] Step 4: Implement deterministic forecast and scenario engine**

Use ranges where data is uncertain. Return confidence and freshness impact. Do not ask an LLM to calculate balances.

- **[] Step 5: Verify and commit**

Run: `pytest tests/meridian/test_beacon.py tests/meridian/test_scenarios.py -q`

```
git add -- meridian/beacon.py meridian/scenarios.py meridian/services tests/meridian
git commit -m "feat: add explainable Meridian scenarios"
```

Task 18: Make the advisor contextual and evidence-bound

Files:

- Create: `meridian/ai/advisor.py`
- Modify: `crew/advisor.py`
- Modify: `meridian/api.py`
- Modify: `static/js/ui/advisor_fab.js`
- Create: `tests/meridian/ai/test_advisor.py`
- Create: `tests/browser/test_contextual_advisor.py`

Interfaces:

- Produces: `AdvisorContext(kind, object_id, evidence_ids)`, evidence-linked answers, global persistence, and proposal-only actions.
- **[] Step 1: Write safe-context and grounding tests**

Open from a transaction, Commitment, forecast, and account. Assert only allowlisted fields enter prompts; numeric claims reference evidence ids; unsupported claims receive an uncertainty response; proposed changes enter the action pipeline.

- **[] Step 2: Run tests and confirm failure**

Run: `pytest tests/meridian/ai/test_advisor.py tests/browser/test_contextual_advisor.py -q`

- **[] Step 3: Implement contextual retrieval and response schema**

Return `{answer, evidence, proposals, provider, model, usage}`. Calculations come from Meridian services; the LLM explains and compares them.

- **[] Step 4: Replace duplicate advisor surfaces**

Keep one globally mounted advisor with desktop inspector and mobile sheet compositions, persistent history, accessible controls, and explicit provider failure states.

- **[] Step 5: Verify and commit**

Run: `pytest tests/meridian/ai tests/browser/test_contextual_advisor.py -q`

```
git add -- meridian/ai/advisor.py crew/advisor.py meridian/api.py static/js/ui/advisor_fab.js tests
git commit -m "feat: ground Meridian advisor in financial evidence"
```

Task 19: Complete Accounts and responsive parity

Files:

- Create: `meridian/services/accounts.py`
- Create: `static/js/meridian/accounts.js`
- Create: `templates/meridian/partials/accounts.html`
- Create: `tests/browser/test_accounts.py`
- Create: `tests/browser/test_responsive_parity.py`

Interfaces:

- Produces unified account, card, family, liability, reimbursement, connection, and freshness views using shared Meridian components.
- **[] Step 1: Write Accounts and parity tests**

Assert Crew, SimpleFin, LunchFlow, Splitwise, cards, and family data use one visual grammar; source marks are subordinate; all desktop actions have mobile routes; and no provider introduces a standalone primary workspace.

- **[] Step 2: Run tests and confirm failure**

Run: `pytest tests/browser/test_accounts.py tests/browser/test_responsive_parity.py -q`

- **[] Step 3: Implement unified Accounts compositions**

Group by owner and financial role rather than provider. Put provider setup, health, and credentials under Connections.

- **[] Step 4: Run accessibility and device matrix**

Verify 390×844, 430×932, 768×1024, 1024×768, and 1440×900; keyboard-only flow; reduced motion; light/dark themes; 200% zoom; and safe-area behavior.

- **[] Step 5: Commit**

```
git add -- meridian/services/accounts.py static/js/meridian/accounts.js
templates/meridian/partials/accounts.html tests/browser
git commit -m "feat: unify Meridian accounts across devices"
```

Task 20: Remove obsolete surfaces after parity audit

Files:

- Modify: `templates/partials/navigation.html`
- Modify: `templates/index.html`
- Modify: `static/js/ui/navigation.js`
- Modify: `static/js/api/expenses.js`
- Modify: `static/js/api/goals.js`

- Modify: static/js/api/account.js
- Modify: static/css/components.css
- Modify: static/css/polish.css
- Create: tests/test_legacy_redirects.py
- Create: docs/MERIDIAN_MIGRATION.md

Interfaces:

- Consumes: completed Meridian parity matrix.
- Produces redirects from old tabs to new workspaces, removal of duplicate advisor/bill/pocket UI, and an auditable migration guide.

- **[] Step 1: Write legacy-route and data-parity tests**

Assert old Bills/Expenses/Pockets/Account entry points land on the correct Meridian workspace and every legacy record is visible through a Commitment, Account, or migration-review entry.

- **[] Step 2: Run tests before removal**

Run: `pytest tests/test_legacy_redirects.py tests/browser/test_responsive_parity.py -q`

- **[] Step 3: Remove duplicate UI and compatibility code only when tests pass**

Delete the Account-page advisor, old Beacon card, duplicate funding manager, and superseded primary navigation. Retain provider/mutation adapters still used by Meridian.

- **[] Step 4: Document upgrade, rollback, and data audit**

Document database backup, migration preview/apply, rollback to the pre-Meridian image, provider freshness verification, and post-upgrade parity checks.

- **[] Step 5: Run final verification**

Run: `ruff check . && pytest -q && docker build -t meridian:release . && pytest tests/browser -q`

Expected: all tests pass with no obsolete primary surfaces and no credential exposure.

- **[] Step 6: Commit**

```
git add -- templates static/js static/css tests/test_legacy_redirects.py docs/MERIDIAN_MIGRATION.md
git commit -m "refactor: complete Meridian workspace migration"
```

- **[] Slice 4 release gate**

Run the complete CI workflow against a sanitized production-data copy, then perform an owner-observed read-only verification against current Crew and outside-account data. Verify transaction totals, owned transfers, credit payments, Splitwise balances, Commitment migration, forecast explanations, mobile parity, and proposal-only mutation behavior before enabling Meridian as the default route.

Slice 5 - Document Intelligence

Task 21: Add encrypted evidence storage and retention

Files:

- Create: meridian/evidence.py
- Create: meridian/migrations/005_evidence_graph.sql
- Create: meridian/storage.py
- Create: tests/meridian/test_evidence.py
- Create: tests/meridian/test_storage.py

Interfaces:

- Produces: EvidenceItem, EvidenceLink, EvidenceRepository, EncryptedBlobStore.put/read/delete, and retention sweeps that preserve audit metadata while deleting expired blobs.

[] Step 1: Write evidence, encryption, deduplication, and retention tests

Assert identical hashes reuse one encrypted blob, plaintext never appears on disk, links retain provenance, revoked-source items become inaccessible, and expiration deletes content without corrupting financial records.

[] Step 2: Run tests and confirm missing-module failures

Run: `pytest tests/meridian/test_evidence.py tests/meridian/test_storage.py -q`

[] Step 3: Implement the evidence graph and blob boundary

Use per-installation envelope encryption backed by a server-side key, SHA-256 content identities, parameterized SQLite operations, explicit MIME/size metadata, and source-scoped revocation. No provider credential enters evidence tables.

[] Step 4: Verify and commit

Run: `pytest tests/meridian/test_evidence.py tests/meridian/test_storage.py -q`

```
git add -- meridian/evidence.py meridian/storage.py meridian/migrations/005_evidence_graph.sql tests/meridian
git commit -m "feat: add Meridian financial evidence storage"
```

Task 22: Ingest read-only email and safe attachments

Files:

- Create: meridian/connectors/email.py
- Create: meridian/documents/safety.py
- Create: meridian/documents/extract.py
- Create: tests/meridian/connectors/test_email.py
- Create: tests/meridian/documents/test_safety.py
- Create: tests/meridian/documents/test_extract.py

Interfaces:

- Produces: ReadOnlyMailConnector.poll(cursor) -> MailBatch, validate_attachment(metadata, stream) -> ValidationResult, and extract_document(blob) -> ExtractedDocument.
- [] Step 1: Write connector scope and cursor tests

Assert the connector requests read-only access, processes each message once, stores source links, handles revocation, and exposes no send/delete/modify methods.

- **[] Step 2: Write attachment safety tests**

Cover allowed PDF/JPEG/PNG types, MIME spoofing, decompression bombs, encrypted documents, size limits, malware-scan rejection, duplicate hashes, and sanitized filenames.

- **[] Step 3: Write deterministic extraction tests**

Use hand-checked bill, statement, receipt, renewal, and pay-stub fixtures. Assert extracted values retain page/region provenance and ambiguous values remain candidates rather than facts.

- **[] Step 4: Run tests and confirm failures**

Run: `pytest tests/meridian/connectors/test_email.py tests/meridian/documents -q`

- **[] Step 5: Implement connector, safety pipeline, and extraction**

Run type, size, malware, hash, and encryption checks before parsing. Use text/OCR extraction before structured AI interpretation. Store only allowlisted metadata and encrypted source blobs.

- **[] Step 6: Verify and commit**

Run: `pytest tests/meridian/connectors tests/meridian/documents -q`

```
git add -- meridian/connectors meridian/documents tests/meridian/connectors tests/meridian/documents
git commit -m "feat: ingest financial email documents safely"
```

Task 23: Reconcile documents with Commitments and transactions

Files:

- Create: `meridian/documents/reconcile.py`
- Modify: `meridian/api.py`
- Modify: `static/js/meridian/transaction-inspector.js`
- Modify: `static/js/meridian/plan.js`
- Create: `tests/meridian/documents/test_reconcile.py`
- Create: `tests/browser/test_document_intelligence.py`

Interfaces:

- Produces: `reconcile_document(extracted, graph) -> DocumentReconciliation`, document-review API, bill->charge->payment chains, and evidence panels in transaction/Commitment details.

- **[] Step 1: Write reconciliation tests**

Cover exact and fuzzy bill matches, statement totals, partial payments, duplicate bills, unexpected price increases, late fees, missing expected bills, and ambiguous chains. Assert no relationship changes transaction totals.

- **[] Step 2: Write browser tests**

Assert source email/document, extracted facts, confidence, discrepancies, PDF access, retention controls, and proposed Commitment/schedule changes appear in existing Review and inspector surfaces.

- **[] Step 3: Run tests and confirm failure**

Run: `pytest tests/meridian/documents/test_reconcile.py tests/browser/test_document_intelligence.py -q`

- **[] Step 4: Implement deterministic matching followed by AI tie-breaking**

Use merchant, amount tolerance, dates, masked account reference, and recurrence first. AI may rank ambiguous candidates but must return evidence ids and confidence. Creating or changing a Commitment remains proposal-only.

- **[] Step 5: Verify and commit**

Run: `pytest tests/meridian/documents tests/browser/test_document_intelligence.py -q`

```
git add -- meridian/documents/reconcile.py meridian/api.py static/js/meridian tests/meridian/documents tests/browser/test_document_intelligence.py
git commit -m "feat: reconcile Meridian financial documents"
```

- **[] Slice 5 release gate**

Run the full gate with synthetic mail fixtures and a malware-test corpus. Verify OAuth revocation, retention deletion, encrypted storage, PDF provenance, and zero external writes.

Slice 6 - Life Context

Task 24: Add opt-in contextual evidence adapters and scenario assumptions

Files:

- Create: `meridian/connectors/calendar.py`
- Create: `meridian/context.py`
- Modify: `meridian/scenarios.py`
- Create: `tests/meridian/connectors/test_calendar.py`
- Create: `tests/meridian/test_context.py`

Interfaces:

- Produces: `ReadOnlyCalendarConnector`, `ContextSignal`, `ContextRepository`, and `scenario_assumptions(signals, graph) -> list[Assumption]`.
- **[] Step 1: Write privacy and inference tests**

Assert source-specific opt-in/revocation, minimum necessary event fields, bounded history, and no expense inference without explicit evidence or user confirmation.

- **[] Step 2: Write context tests for payroll, travel, and shared money**

Cover pay-stub/deposit mismatch, dated travel pressure, temporary trip scenarios, school/household seasonality, and Splitwise reimbursement timing. Every assumption must expose source, confidence, range, and confirmation state.

- **[] Step 3: Run tests and confirm failure**

Run: `pytest tests/meridian/connectors/test_calendar.py tests/meridian/test_context.py -q`

- **[] Step 4: Implement read-only adapters and assumption generation**

Keep event descriptions out of AI prompts unless selected by an allowlisted extractor. Treat correlations as hypotheses and never persist scenario changes without approval.

- **[] Step 5: Verify and commit**

Run: `pytest tests/meridian/connectors/test_calendar.py tests/meridian/test_context.py tests/meridian/test_scenarios.py -q`

```
git add -- meridian/connectors/calendar.py meridian/context.py meridian/scenarios.py tests/meridian
git commit -m "feat: add opt-in life context to Meridian"
```

- **[] Slice 6 release gate**

Verify every context source can be independently revoked, removing it recomputes affected assumptions, and Today/Plan label context-driven scenarios as assumptions rather than facts.

Slice 7 - Asset and Contract Memory

Task 25: Model assets, contracts, warranties, and obligations

Files:

- Create: `meridian/assets.py`
- Create: `meridian/contracts.py`
- Create: `meridian/migrations/006_assets_contracts.sql`
- Create: `tests/meridian/test_assets.py`
- Create: `tests/meridian/test_contracts.py`

Interfaces:

- Produces: Asset, Contract, Warranty, Obligation, return/renewal/maintenance events, and links to evidence, transactions, and Commitments.
- **[] Step 1: Write lifecycle and provenance tests**

Cover return windows, warranty expiration, renewal/cancellation dates, escalation clauses, deductibles, maintenance intervals, replacement reserves, and source-document corrections.

- **[] Step 2: Write advisory-boundary tests**

Medical, insurance, lease, and tax-related documents may yield quoted financial facts and deadlines but must not yield medical, legal, coverage, or tax determinations.

- **[] Step 3: Run tests and confirm failure**

Run: `pytest tests/meridian/test_assets.py tests/meridian/test_contracts.py -q`

- **[] Step 4: Implement models and repositories**

Store extracted facts with evidence spans and confidence. Generate proposed Commitment or reminder changes rather than applying them directly.

- **[] Step 5: Verify and commit**

Run: `pytest tests/meridian/test_assets.py tests/meridian/test_contracts.py -q`

```
git add -- meridian/assets.py meridian/contracts.py meridian/migrations/006_assets_contracts.sql tests/meridian
git commit -m "feat: add Meridian asset and contract memory"
```

Task 26: Integrate evidence memory into Today, Plan, Activity, and Accounts

Files:

- Create: meridian/services/memory.py
- Modify: meridian/services/today.py
- Modify: meridian/services/plan.py
- Modify: meridian/services/activity.py
- Modify: meridian/services/accounts.py
- Modify: meridian/api.py
- Create: static/js/meridian/memory.js
- Create: tests/browser/test_evidence_memory.py

Interfaces:

- Produces: return/renewal/warranty/maintenance attention items, evidence-linked reserve scenarios, and asset/contract drill-downs using the existing visual system.

- **[] Step 1: Write cross-workspace browser tests**

Assert relevant evidence appears in existing workspaces without adding a fifth primary navigation item, preserves Editorial Wealth styling, links to source, supports mobile parity, and explains why each item matters financially.

- **[] Step 2: Run tests and confirm failure**

Run: `pytest tests/browser/test_evidence_memory.py -q`

- **[] Step 3: Implement memory service and compositions**

Today shows dated attention; Plan shows reserve effects; Activity links receipts/documents; Accounts holds asset and contract structure. Reuse inspector, review, and advisor patterns.

- **[] Step 4: Verify and commit**

Run: `pytest tests/meridian tests/browser/test_evidence_memory.py -q`

```
git add -- meridian/services static/js/meridian/memory.js meridian/api.py tests/browser/test_evidence_memory.py
git commit -m "feat: connect Meridian evidence memory across workspaces"
```

- **[] Final evidence-graph release gate**

Run full CI, privacy and revocation tests, a document-security corpus, cross-provider reconciliation, accessibility/device matrices, and an owner-observed read-only verification. Confirm evidence deletion does not corrupt financial history and no source grants write access.